

STUDENT EXPENSE MANAGER

VITyarthi - Build Your Own Project

Student Expense Manager

Submitted By:

Prashant Patil

Course:

B.Tech - Artificial Intelligence and Machine Learning

School:

SCAI

Technology:

Java

Academic Year:

2026-27

2. Introduction

Managing daily expenses is important for students because small expenses such as food, transportation, stationery and other personal requirements can accumulate over time.

The **Student Expense Manager** is a Java-based console application designed to help students record and manage their daily expenses. The application provides features to add, view, search and delete expenses. It can also generate monthly summaries and category-wise expense information.

The project uses Java concepts such as Object-Oriented Programming, collections, interfaces, exception handling and file handling. The application does not use a database. Expense information is stored in a text file.

3. Problem Statement

Students often make several small payments during their daily activities and may find it difficult to keep track of their total spending.

A simple system is required that allows students to record their expenses, view previous records, search for specific expenses, delete incorrect records and calculate monthly spending.

The Student Expense Manager provides a simple console-based solution for managing these expense records.

4. Objectives

The main objectives of the project are:

1. To develop a simple Java-based expense management application.
2. To allow students to record their daily expenses.
3. To provide a facility to view all recorded expenses.
4. To allow users to search expenses by category or description.
5. To allow users to delete expenses using a unique ID.
6. To calculate monthly total expenses.
7. To display category-wise expense information.
8. To save and load expense records using file handling.
9. To demonstrate practical application of Java programming concepts.

5. Scope of the Project

The project is designed for students who want a simple way to manage their daily expenses.

The system supports:

- Adding expenses
- Viewing expenses
- Searching expenses
- Deleting expenses
- Monthly expense summaries
- Category-wise expense calculation
- Saving expense records
- Loading previously saved records
- Basic input validation

The project is implemented as a **console application** and does not require a database or external server.

6. Target Users

The primary target users are:

- College students
- Hostel students
- Students managing personal budgets
- Users who need a simple expense tracking system

7. Functional Requirements

The project contains the following functional modules:

7.1 Expense Management

The user can add a new expense by entering:

- Category
- Description
- Amount
- Date

Each expense receives a unique ID.

7.2 Expense Viewing and Searching

The user can:

- View all stored expenses.
- Search expenses using category.
- Search expenses using description.

7.3 Expense Deletion

The user can delete an expense by entering its unique expense ID.

7.4 Monthly Reporting

The system calculates:

- Number of expenses in a selected month
- Total monthly expense
- Category-wise expense totals

7.5 File Storage

The system allows users to:

- Save expense records to a text file.
- Load previously saved expense records.

8. Non-Functional Requirements

The project follows the following non-functional requirements. The VITyarthi guidelines require at least four such requirements.

8.1 Usability

The application uses a simple menu-driven console interface so that users can easily select different operations.

8.2 Reliability

The application provides exception handling and input validation to reduce unexpected program failures.

8.3 Maintainability

The project is divided into separate classes such as Expense, ExpenseManager, FileManager, Main and Report.

8.4 Performance

The application uses Java collections such as ArrayList and HashMap for efficient temporary data management.

8.5 Error Handling

Invalid menu choices and invalid numeric inputs are handled using exception handling.

8.6 Resource Efficiency

The project uses a simple text file for storage and does not require a database or external server.

9. System Architecture

Architecture Description

- **Main** handles user interaction and displays the menu.
- **Expense** represents an individual expense.
- **ExpenseManager** manages the collection of expenses and performs operations.
- **FileManager** handles saving and loading expense data.
- **Report** defines the monthly summary operation.
- **expenses.txt** is used for permanent text-based storage.

10. Use Case Diagram

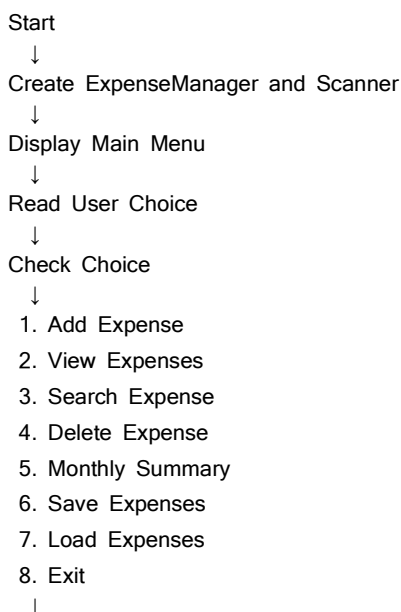
Use Cases

The user can:

- Add Expense
- View Expenses
- Search Expense
- Delete Expense
- Generate Monthly Summary
- Save Expenses
- Load Expenses
- Exit Application

11. Workflow Diagram

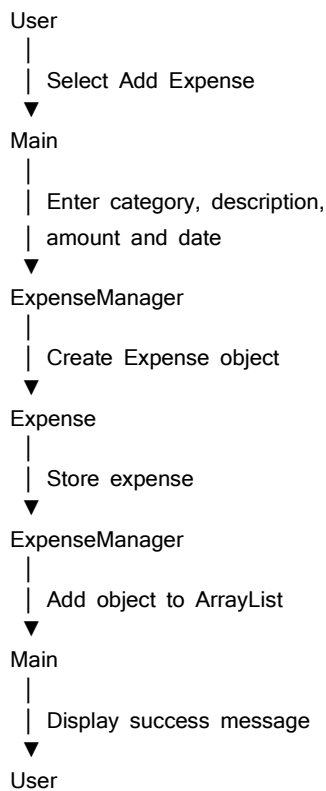
The overall workflow of the application is:



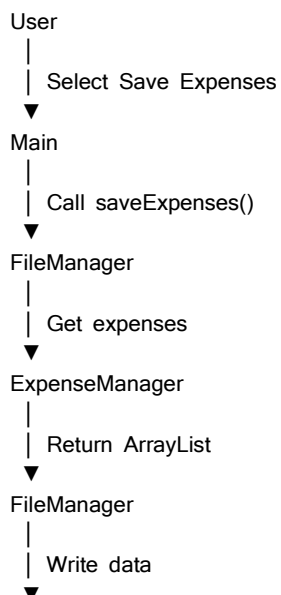
Perform Selected Operation
↓
Return to Main Menu
↓
Choice = 8?
↓
Yes → End

12. Sequence Diagram

Add Expense Sequence



Save Expense Sequence



13. Class Diagram

The project contains the following main classes and interface:

Expense

Stores information about an individual expense.

ExpenseManager

Manages the collection of expenses and provides operations such as add, search, delete and monthly summary.

FileManager

Handles file-based saving and loading.

Main

Provides the menu-driven user interface.

Report

An interface containing the `monthlySummary()` method.

14. Storage Design

The project does not use a relational database.

Expense records are stored in:

data/expenses.txt

Each record contains:

ID | Category | Description | Amount | Date

Example:

```
1|Food|Lunch|120.0|15-09-2026
2|Travel|Bus ticket|40.0|15-09-2026
```

The `FileManager` class is responsible for reading and writing these records.

15. Design Decisions and Rationale

Use of ArrayList

ArrayList is used to store multiple Expense objects because the number of expenses can change during program execution.

Use of HashMap

HashMap is used in the monthly summary to calculate category-wise expense totals.

Use of Iterator

Iterator is used while deleting an expense from the ArrayList.

Use of Interface

The Report interface defines the monthlySummary() operation and demonstrates interface-based design.

Use of File Handling

Text-file storage was selected because the project is intended to remain simple and does not require database infrastructure.

Use of Exception Handling

Exception handling is used to manage invalid user inputs and file-related errors.

16. Implementation Details

The project is implemented using **Java 21**.

Main.java

Handles:

- Menu display
- User input
- Menu selection
- Input validation
- Calling appropriate methods

Expense.java

Represents an individual expense and contains:

- ID
- Category
- Description
- Amount
- Date

ExpenseManager.java

Provides:

- Add expense
- View expenses
- Search expense
- Delete expense
- Monthly summary
- Expense collection management

FileManager.java

Provides:

- Saving expenses
- Loading expenses
- Text-file handling

Report.java

Defines the monthly summary method.

17. Java Concepts Used

The project demonstrates the following Java concepts:

- Classes and Objects
- Constructors
- Encapsulation
- Private data members
- Getter and Setter methods
- this keyword
- Methods
- Interfaces
- ArrayList

- HashMap
- Iterator
- For loops
- While loops
- Do-while loop
- Switch statement
- Conditional statements
- Exception handling
- File input/output
- BufferedReader
- BufferedWriter

These concepts are applied in a practical project rather than being used only as isolated examples.

18. Screenshots / Results

```
STUDENT EXPENSE MANAGER
1. Add Expense
2. View Expenses
3. Search Expense
4. Delete Expense
5. Monthly Summary
6. Save Expenses
7. Load Expenses
8. Exit
Enter your choice: 4

Enter expense ID: 1
Expense deleted successfully.
```

```
Expense added successfully.
STUDENT EXPENSE MANAGER
1. Add Expense
2. View Expenses
3. Search Expense
4. Delete Expense
5. Monthly Summary
6. Save Expenses
7. Load Expenses
8. Exit
Enter your choice: 4

Enter expense ID: 1
```

```
STUDENT EXPENSE MANAGER
1. Add Expense
2. View Expenses
3. Search Expense
4. Delete Expense
5. Monthly Summary
6. Save Expenses
7. Load Expenses
8. Exit
Enter your choice: 1

--- Add Expense ---
Enter category: food
Enter description: lunch
Enter amount: 543
Enter date (DD-MM-YYYY): 17-09-2026
Expense added successfully!
```

```
>> java -cp src Main
STUDENT EXPENSE MANAGER
1. Add Expense
2. View Expenses
3. Search Expense
4. Delete Expense
5. Monthly Summary
6. Save Expenses
7. Load Expenses
8. Exit
Enter your choice: 1

--- Add Expense ---
Enter category: food
Enter description: lunch
Enter amount: 120
Enter date (DD-MM-YYYY): 17-9-26
Expense added successfully!
STUDENT EXPENSE MANAGER
1. Add Expense
2. View Expenses
3. Search Expense
4. Delete Expense
5. Monthly Summary
6. Save Expenses
7. Load Expenses
8. Exit
```

19. Testing Approach

The application was tested using different user inputs and operations.

Test Case	Input/Action	Expected Result
TC01	Select Add Expense	Expense is added
TC02	Enter amount ≤ 0	Error message displayed
TC03	Select View Expenses	All expenses displayed
TC04	Search existing category	Matching expenses displayed
TC05	Search unavailable category	No matching expense message
TC06	Delete existing ID	Expense deleted
TC07	Delete invalid ID	Not found message
TC08	Generate monthly summary	Monthly total displayed
TC09	Save expenses	Data saved to file
TC10	Load expenses	Saved data loaded
TC11	Invalid menu choice	Invalid choice message
TC12	Invalid numeric input	Input error handled

20. Challenges Faced

During development, some challenges were encountered.

20.1 Understanding Java Collections

Using ArrayList, HashMap and Iterator required understanding how Java collections work.

20.2 File Handling

Saving and loading structured data from a text file required proper conversion between strings and numeric values.

20.3 Input Handling

The application accepts both text and numeric input through Scanner, so input handling needed proper validation.

20.4 Modular Design

The project was divided into different classes to keep the implementation organized and maintainable.

21. Learnings and Key Takeaways

Through this project, the following concepts were learned and applied:

1. How to design a small real-world Java application.
2. How to use Object-Oriented Programming.
3. How to create and use classes and objects.
4. How encapsulation works using private variables and methods.
5. How to use Java Collections.
6. How to implement an interface.
7. How to perform file input/output.
8. How to handle invalid user input using exceptions.
9. How to divide a project into multiple modules.
10. How to document and test a software project.

The project helped connect theoretical Java concepts with practical application development.

22. Future Enhancements

The project can be improved in the future by adding:

- Graphical User Interface
- User login system
- Budget limits and alerts
- Expense charts
- CSV export
- More advanced reporting
- Date-based filtering
- Database integration
- Cloud-based storage
- Mobile or web interface

23. Conclusion

The **Student Expense Manager** is a simple Java console application that provides essential functionality for recording and managing student expenses.

The project successfully demonstrates Java programming concepts including Object-Oriented Programming, collections, interfaces, exception handling and file handling.

The application provides a modular structure with separate classes for expense management, file handling, user interaction and reporting. It also provides basic validation and persistent storage through a text file.

Overall, the project demonstrates how Java concepts can be applied to develop a practical real-world application.

24. References

1. VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines.
2. Java Documentation - Java Programming Language.
3. Java Collections Framework documentation.
4. Java File I/O documentation.
5. Course lecture notes and learning materials.